Programming, Data Structures & Algorithms

Revision Sheet

Every topic on its own page • method, complexity, worked example, traps

How to use this sheet

- **One topic per page.** Nothing splits across pages — read a page, close the book, and try to reproduce the formula box from memory.
- **Read in this order on each page:** the italic line (what the topic *is*) → the formula box → "how to approach" → the worked example → the traps.
- **The dashed "My notes" box is yours.** Add the variant that caught you out, a formula your coaching notes state differently, or the page number of a question you got wrong.
- **Traps in the orange box are the marks people actually lose.** Re-read only those the night before the exam.

Contents

1. Python Language Semantics
2. Built-in Data Structures
3. Recursion & Code Tracing
4. Asymptotic Complexity & Recurrences
5. Searching & Sorting
6. Stacks, Queues & Linked Lists
7. Trees & Binary Search Trees
8. Hashing
9. Graphs & Graph Algorithms
10. Divide & Conquer and Algorithm Design

1. Python Language Semantics

GATE DA tests Python because Python has traps. Almost every 'what does this print?' question is one of the four below.

KEY FORMULAS

The mutable default argument

`def f(x, L=[]):` — L is created ONCE, at definition

Aliasing vs copying

`b = a` (same list) `b = a[:]` (a copy)

WHAT YOU MUST KNOW

- **Mutable default**: the list persists between calls, so it *accumulates*. Passing an explicit list resets it.
- Lists/dicts/sets are passed **by reference**: mutating inside a function is visible outside; *rebinding* (`L = [...]`) is not.
- A **closure** captures the *variable*, not its value at definition time; two calls to the outer function make two separate variables.
- `a, b = b, a` evaluates the whole right side first, so it is a true simultaneous swap.

HOW TO APPROACH A QUESTION

1. Scan for `=[]` or `={}` in a parameter list — that is the trap being tested.
2. Ask for each call: does this *mutate* the object or *rebind* the name?
3. Write the object's contents on paper after every single call — never trace in your head.
4. For closures, draw one box per call to the outer function.

WORKED EXAMPLE

Q. What does this print?

```
def f(v, L=[]):  L.append(v);  return L
print(f(1)); print(f(2)); print(f(3, []))
```

Calls 1 and 2 share the *same* default list; call 3 supplies a fresh one:

`[1]` `[1, 2]` `[3]`

WATCH OUT

- Expecting the default list to reset each call — it does not.
- Thinking `L = L + [x]` mutates; it rebinds a local name and the caller sees nothing.

MY NOTES

2. Built-in Data Structures

Know the cost of each operation. Most 'which is faster' questions are answered from one table.

KEY FORMULAS

list

index $O(1)$, append $O(1)^*$, insert/remove $O(n)$, `in` $O(n)$

dict / set (hashing)

lookup, insert, delete $O(1)$ average, $O(n)$ worst

sorting

`sorted(L)` $O(n \log n)$

WHAT YOU MUST KNOW

- `tuple` is immutable and hashable, so it can be a dict key; a `list` cannot.
- Slicing `L[i:j]` **creates a copy**; `L[::-1]` returns a reversed copy.
- `len, sum, min, max` are all $O(n)$ — calling them inside a loop makes it $O(n^2)$.
- Set membership is $O(1)$; converting a list to a set before repeated lookups is the standard speed-up.

HOW TO APPROACH A QUESTION

1. Identify the operation being repeated inside the loop and multiply its cost by the loop count.
2. Ask whether the structure needs *order* (list), *uniqueness* (set) or *key lookup* (dict).
3. Watch for hidden copies from slicing inside a loop.

WORKED EXAMPLE

Q. What is the complexity of checking membership of each of n items against a list of size m?

$$n \times O(m) = O(nm) \quad \text{; converting the list to a set first gives } O(n+m)$$

WATCH OUT

- Assuming `in` on a list is fast — it is a linear scan.
- Forgetting that a slice copies, which turns an $O(n)$ loop into $O(n^2)$.

MY NOTES

3. Recursion & Code Tracing

Draw the call tree. Every recursion question — count of calls, output, depth — is answered by the same drawing.

KEY FORMULAS

Naive Fibonacci call count

$$\text{leaf calls for } F(n) = F(n{+}1)$$

Total activations

$$T(n) = T(n{-}1) + T(n{-}2) + 1$$

WHAT YOU MUST KNOW

- Each call is pushed on the **runtime stack** and popped only when it returns — "activations" means total calls, including the first.
- The base case fires first and returns upward; work *after* the recursive call happens on the way back.
- Recursion depth = the longest root-to-leaf path, not the number of calls.
- A recursion that mutates a list passed in changes the caller's list too.

HOW TO APPROACH A QUESTION

1. Draw the call tree with the argument written in each node.
2. Label each leaf with its return value, then combine upward.
3. To count activations, count *nodes*; to count base cases, count *leaves*.
4. For small n, just tabulate $f(0), f(1), f(2), \ldots$ and find the pattern.

WORKED EXAMPLE

Q. `mystery(n)` returns 1 if $n \le 0$, else `mystery(n-1)+mystery(n-2)`. Total calls for `mystery(4)`?
Let $T(n)$ be total activations: $T(0) = T(-1) = 1$ and $T(n) = 1 + T(n{-}1) + T(n{-}2)$.

$$T(1) = 3,\ T(2) = 5,\ T(3) = 9,\ T(4) = 1 + 9 + 5 = \mathbf{15}$$

WATCH OUT

- Counting only the leaves when the question asks for total calls.
- Forgetting the initial call in the count.

MY NOTES

4. Asymptotic Complexity & Recurrences

You are asked to recognise a recurrence, not to prove it. Master theorem plus two standard patterns cover almost everything.

KEY FORMULAS

Growth order

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$$

Master theorem $T(n) = aT(n/b) + \Theta(n^d)$

$$\begin{cases} \Theta(n^d) & d > \log_b a \\ \Theta(n^d \log n) & d = \log_b a \\ \Theta(n^{\log_b a}) & d < \log_b a \end{cases}$$

WHAT YOU MUST KNOW

- $T(n) = 2T(n/2) + O(n) \Rightarrow \Theta(n \log n)$ — merge sort, balanced quicksort.
- $T(n) = T(n-1) + O(n) \Rightarrow \Theta(n^2)$ — worst-case quicksort.
- $T(n) = T(n/2) + O(1) \Rightarrow \Theta(\log n)$ — binary search.
- Big-O is an *upper* bound; Θ is tight. A question saying "exactly" wants Θ.

HOW TO APPROACH A QUESTION

1. Read off a (number of subproblems), b (shrink factor) and d (combine cost exponent).
2. Compare d with $\log_b a$ and pick the case.
3. If the split is $n-1$ rather than n/b, the master theorem does *not* apply — expand the sum instead.

WORKED EXAMPLE

Q. Solve $T(n) = 2T(n/2) + \Theta(n)$.

$$a = 2,\ b = 2,\ d = 1; \quad \log_b a = \log_2 2 = 1 = d$$

$$\Rightarrow\ T(n) = \boldsymbol{\Theta(n \log n)}$$

WATCH OUT

- Applying the master theorem to $T(n) = T(n-1) + \ldots$ (subtractive, not divide-and-conquer).
- Confusing worst case with average case for quicksort.

MY NOTES

5. Searching & Sorting

GATE DA asks for counts: how many comparisons, how many swaps, what does the array look like after two passes.

KEY FORMULAS

Binary search (worst case)

$$\lfloor \log_2 n \rfloor + 1 \text{ comparisons}; \quad n = 1000 \Rightarrow \mathbf{10}$$

Comparison-sort lower bound

$$\Omega(n \log n)$$

WHAT YOU MUST KNOW

- **Merge sort** $\Theta(n \log n)$ always, stable, needs $O(n)$ extra space.
- **Quicksort** average $\Theta(n \log n)$, worst $\Theta(n^2)$ (already-sorted input with an end pivot), in place.
- **Bubble/Insertion/Selection** are $\Theta(n^2)$; insertion is $O(n)$ on nearly-sorted data.
- After k passes, **selection sort** has the first k positions final; **bubble sort** has the last k final.

HOW TO APPROACH A QUESTION

1. Write the array out and perform the passes literally — these are counting questions.
2. For binary search, halve the interval and count comparisons, not iterations.
3. For "after exactly two passes", check each algorithm separately; the answer is rarely the same for all.

WORKED EXAMPLE

Q. Which of bubble / insertion / selection sorts $[4, 3, 2, 1, 5]$ into increasing order after *exactly two* passes?
Bubble: pass 1 $\rightarrow [3, 2, 1, 4, 5]$, pass 2 $\rightarrow [2, 1, 3, 4, 5]$ — not sorted. Insertion: pass 1 $\rightarrow [3, 4, 2, 1, 5]$, pass 2 $\rightarrow [2, 3, 4, 1, 5]$ — not sorted.
Selection: pass 1 puts the minimum first $\rightarrow [1, 3, 2, 4, 5]$; pass 2 puts the next minimum second $\rightarrow [1, 2, 3, 4, 5]$.

Selection sort only

WATCH OUT

- Assuming quicksort is always $n \log n$ — sorted input with a bad pivot is n^2.
- Counting iterations instead of comparisons in binary search.

MY NOTES

6. Stacks, Queues & Linked Lists

Simple structures, tested through behaviour: given this sequence of operations, what is the final state?

KEY FORMULAS

Stack (LIFO)

$$\text{push, pop, top all } O(1)$$

Queue (FIFO)

$$\text{enqueue, dequeue } O(1)$$

Singly linked list

$$\text{insert/delete at a known node } O(1),\ \text{search } O(n)$$

WHAT YOU MUST KNOW

- Function calls, DFS, undo and expression evaluation all use a **stack**; BFS uses a **queue**.
- A linked list has **no random access** — reaching index k costs $O(k)$.
- Deleting a node needs a pointer to its *predecessor* (unless the list is doubly linked).
- A deque supports $O(1)$ insertion and removal at both ends.

HOW TO APPROACH A QUESTION

1. Draw the structure and update it operation by operation — do not shortcut.
2. For stacks, remember the *last* thing in is the first out; write the contents vertically.
3. For linked-list pointer questions, draw the arrows before and after each step.

WORKED EXAMPLE

Q. Push 1,2,3; pop; push 4; pop; pop. What is popped, in order?
Stack after pushes: $[1, 2, 3]$ (3 on top). Pop $\to 3$. Push $4 \to [1, 2, 4]$. Pop $\to 4$. Pop $\to 2$.

$$\text{popped order} = \mathbf{3},\ \mathbf{4},\ \mathbf{2}$$

WATCH OUT

- Treating a stack as FIFO (that is a queue).
- Losing the rest of the list by reassigning a pointer in the wrong order.

MY NOTES

7. Trees & Binary Search Trees

One fact does most of the work: an inorder traversal of a BST comes out sorted.

KEY FORMULAS

BST property

$$\text{left} < \text{node} < \text{right}$$

Operations

$$\text{search/insert/delete} = O(h);\ h = \Theta(\log n) \text{ balanced},\ \Theta(n) \text{ skewed}$$

Traversals

$$\text{pre} = N\,L\,R, \quad \text{in} = L\,N\,R, \quad \text{post} = L\,R\,N$$

WHAT YOU MUST KNOW

- **Inorder of a BST is the sorted order** — use it to check any traversal answer instantly.
- A binary tree of n nodes has height between $\lceil \log_2(n+1) \rceil - 1$ and $n-1$.
- A tree with n nodes has exactly $n-1$ edges.
- Inserting sorted data into a plain BST produces a skewed tree — the $O(n)$ worst case.

HOW TO APPROACH A QUESTION

1. Draw the tree by inserting the keys in the given order.
2. For any traversal question, write the inorder first — it must be sorted, which validates your tree.
3. For deletion, replace with the inorder predecessor or successor.

WORKED EXAMPLE

Q. Insert $50, 30, 70, 20, 40, 60, 80$ into a BST. Give the preorder.

50 is the root; 30 and its children 20, 40 on the left; 70 with 60, 80 on the right.

$$\text{preorder} = \mathbf{50}, \mathbf{30}, \mathbf{20}, \mathbf{40}, \mathbf{70}, \mathbf{60}, \mathbf{80}$$

Check: inorder gives $20, 30, 40, 50, 60, 70, 80$ — sorted, so the tree is right.

WATCH OUT

- Mixing up preorder and postorder — the root is first vs last.
- Assuming a BST is automatically balanced.

MY NOTES

8. Hashing

Average O(1), worst O(n). Questions test collision handling and what the table looks like afterwards.

KEY FORMULAS

Load factor

$$\alpha = \frac{n}{m} \quad (n \text{ keys},\ m \text{ slots})$$

Linear probing

$$h_i(k) = \big(h(k) + i\big) \bmod m$$

Unsuccessful search (open addressing)

$$\approx \frac{1}{1-\alpha} \text{ probes}$$

WHAT YOU MUST KNOW

- **Chaining**: each slot holds a list; average search $O(1+\alpha)$, worst $O(n)$ if all keys collide.
- **Open addressing**: probe until a free slot; the table can never hold more than m keys.
- Deletion in open addressing needs a *tombstone* marker, or later searches break.
- A good hash spreads keys uniformly; a bad one degrades the table to a linked list.

HOW TO APPROACH A QUESTION

1. Compute $h(k)$ for each key in the given insertion order — order matters.
2. On a collision, apply the stated probing rule step by step and write the table out.
3. Count probes as the number of slots *examined*, including the first.

WORKED EXAMPLE

Q. $m = 7$, $h(k) = k \bmod 7$, linear probing. Insert $10, 17, 24$.
$10 \bmod 7 = 3 \to$ slot 3. $17 \bmod 7 = 3$ occupied $\to$ slot 4. $24 \bmod 7 = 3$ occupied, 4 occupied $\to$ slot 5.

slots $3, 4, 5$ filled; 24 took **3** probes

WATCH OUT

- Forgetting that insertion order changes the final table.
- Deleting by simply blanking a slot in open addressing.

MY NOTES

9. Graphs & Graph Algorithms

Know which algorithm solves which problem, and its cost. That single table answers most graph MCQs.

KEY FORMULAS

Traversal

$$\text{BFS, DFS} = O(V + E)$$

Shortest paths

$$\text{Dijkstra (no negative) } O((V+E)\log V); \quad \text{Bellman–Ford } O(VE)$$

Minimum spanning tree

$$\text{Kruskal, Prim } O(E \log V)$$

WHAT YOU MUST KNOW

- **BFS gives the shortest path in an unweighted graph**; DFS does not.
- A tree on V vertices has exactly $V - 1$ edges; a complete graph has $\binom{V}{2}$.
- Topological sort exists *only* for a DAG; it detects cycles as a side effect.
- Bellman–Ford handles negative weights and reports negative cycles; Dijkstra breaks on them.

HOW TO APPROACH A QUESTION

1. Classify: weighted or not? negative weights? directed? cyclic?
2. Pick the algorithm from that classification, then quote its complexity.
3. For a hand-simulation, maintain the frontier (queue for BFS, stack for DFS, heap for Dijkstra) explicitly.

WORKED EXAMPLE

Q. Which algorithm finds shortest paths from one source in a directed graph with some negative edges?
Dijkstra is invalid with negative weights, so:

$$\textbf{Bellman–Ford}, \quad O(VE)$$

WATCH OUT

- Using Dijkstra when negative edges are present.
- Assuming DFS finds shortest paths.

MY NOTES

10. Divide & Conquer and Algorithm Design

Four paradigms. Recognising which one a problem needs is the whole question.

KEY FORMULAS

Divide and conquer

$$T(n) = aT(n/b) + \text{combine cost}$$

Dynamic programming cost

$$T = (\#\text{ states}) \times (\text{work per state})$$

WHAT YOU MUST KNOW

- **Divide & conquer**: independent subproblems (merge sort, binary search).
- **Dynamic programming**: *overlapping* subproblems + optimal substructure — memoise (Fibonacci, knapsack, LCS).
- **Greedy**: one locally best choice, never reconsidered (Kruskal, Prim, Dijkstra, Huffman). Needs a proof of correctness.
- **Backtracking**: build partial solutions and prune (N-queens, subsets).

HOW TO APPROACH A QUESTION

1. Ask: do the subproblems *repeat*? If yes it is DP, not plain divide and conquer.
2. Ask: does a locally optimal choice stay optimal? If yes, greedy may work.
3. For DP, count the states and the work per state to get the complexity.

WORKED EXAMPLE

Q. Why is naive recursive Fibonacci exponential while the DP version is linear?

The recursion recomputes the same $F(k)$ exponentially many times; memoising makes each of the n states cost $O(1)$:

$$O(2^n) \longrightarrow O(n)$$

WATCH OUT

- Calling something greedy when it needs DP — greedy fails on 0/1 knapsack.
- Forgetting that DP needs both overlapping subproblems *and* optimal substructure.

MY NOTES